

1. What are Aggregate Functions?

Aggregate functions are SQL functions that perform a calculation on **multiple rows** and return **one result**.

For example, suppose the **customer** table contains 599 customers.

If we want to know:

- How many customers are there?
- What is the average customer ID?
- What is the highest customer ID?
- What is the lowest customer ID?

We can use aggregate functions.

Common Aggregate Functions

Function	Purpose	Example
COUNT()	Counts rows/values	Number of customers
SUM()	Adds numeric values	Total payment
AVG()	Calculates average	Average payment
MAX()	Finds highest value	Highest payment
MIN()	Finds lowest value	Lowest payment

2. Why Do We Use Aggregate Functions?

Without aggregate functions, SQL normally returns **individual rows**.

For example:

```
SELECT amount
FROM payment;
```

This gives us many payment amounts:

```
2.99
4.99
7.99
2.99
...
```

But sometimes we need a **summary** instead:

```
"How much money was collected in total?"
```

We can use:

```
SELECT SUM(amount)
FROM payment;
```

Result:

```
67416.51
```

So, aggregate functions are mainly used for **summary and analysis of data**.

3. COUNT()

`COUNT()` is used to count records.

Count all customers

```
SELECT COUNT(*)
FROM customer;
```

Meaning

```
COUNT(*) → Count all rows
```

Example result:

```
599
```

There are 599 customers in the `customer` table.

Give the result a meaningful name

We can use an alias:

```
SELECT COUNT(*) AS total_customers
FROM customer;
```

Result:

total_customers

599

4. COUNT(column)

We can also count values in a particular column.

```
SELECT COUNT(email)
FROM customer;
```

Important:

COUNT(column) does **not** count NULL values.

Whereas:

```
COUNT(*)
```

counts all rows.

Remember

```
COUNT(*)      → counts rows
COUNT(column) → counts non-NULL values
```

5. SUM()

SUM() adds numeric values.

The payment table contains a column called amount.

To calculate the total amount of all payments:

```
SELECT SUM(amount) AS total_payment
FROM payment;
```

Example result:

total_payment

67416.51

Real-life meaning

This answers:

"How much total payment has been received?"

6. AVG()

AVG() calculates the average.

For example:

```
SELECT AVG(amount) AS average_payment
FROM payment;
```

Example result:

average_payment

4.20

This answers:

"What is the average payment amount?"

Making the result easier to read

We can round the result:

```
SELECT ROUND(AVG(amount), 2) AS average_payment
FROM payment;
```

Example:

4.20

7. MAX()

MAX() finds the highest value.

```
SELECT MAX(amount) AS highest_payment
FROM payment;
```

Example result:

highest_payment

11.99

This answers:

"What is the largest payment?"

8. MIN()

MIN() finds the smallest value.

```
SELECT MIN(amount) AS lowest_payment
FROM payment;
```

Example result:

lowest_payment

0.99

9. Using Multiple Aggregate Functions

We can use several aggregate functions in one query.

```
SELECT
  COUNT(*) AS total_payments,
  SUM(amount) AS total_amount,
  AVG(amount) AS average_amount,
  MAX(amount) AS highest_payment,
  MIN(amount) AS lowest_payment
FROM payment;
```

This produces a summary such as:

total_payments	total_amount	average_amount	highest_payment	lowest_payment
14596	67416.51	4.20	11.99	0.99

Important idea

One SQL query can summarize **thousands of rows into one row**.

10. Aggregate Functions with WHERE

We can first filter the records and then calculate the aggregate.

For example, find the total payments greater than 5:

```
SELECT SUM(amount) AS total_payment
FROM payment
WHERE amount > 5;
```

The process is:

```
payment table
  ↓
WHERE amount > 5
  ↓
SUM(amount)
  ↓
one result
```

11. COUNT with WHERE

How many payments are greater than 5?

```
SELECT COUNT(*) AS payments_above_5
FROM payment
WHERE amount > 5;
```

12. Aggregate Functions with Dates

The `payment` table contains `payment_date`.

Suppose we want to calculate the total payment made in 2007:

```
SELECT SUM(amount) AS total_payment
FROM payment
WHERE payment_date >= '2007-01-01'
      AND payment_date < '2008-01-01';
```

This is useful when analyzing sales or payments over a particular period.

13. GROUP BY — The Next Important Concept

What is **GROUP BY** in SQL?

GROUP BY is used to **combine rows with the same value into groups** so that we can calculate a summary for each group using aggregate functions such as:

- **COUNT()** → how many?
- **SUM()** → total?
- **AVG()** → average?
- **MAX()** → highest?
- **MIN()** → lowest?

Simple example from **dvdrental**

Suppose we want to know:

How many payments has each customer made?

```
SELECT
    customer_id,
    COUNT(*) AS total_payments
FROM payment
GROUP BY customer_id;
```

What does it do?

Without **GROUP BY**, SQL counts **all payments**:

```
SELECT COUNT(*)
FROM payment;
```

Result:

```
14596
```

But with:

```
GROUP BY customer_id
```

SQL creates a separate group for each customer:

```
Customer 1 → all payments of customer 1
Customer 2 → all payments of customer 2
```

Customer 3 → all payments of customer 3
...

Then `COUNT()` counts the payments **inside each group**.

Result:

customer_id	total_payments
1	32
2	27
3	26
4	22
...	...

Another example

How much has each customer paid in total?

```
SELECT
  customer_id,
  SUM(amount) AS total_payment
FROM payment
GROUP BY customer_id;
```

So, the main idea is:

GROUP BY divides rows into groups based on a column, allowing us to calculate a summary for each group.

Easy memory trick

GROUP BY = "Give me the summary for each..."

For example:

- total payment **for each customer**
- number of films **for each rating**
- rentals **for each customer**
- payments **for each staff member**

When you see **"each"**, **"per"**, or **"by"** in a summary question, think about **GROUP BY**.

14. COUNT with GROUP BY

How many payments has each customer made?

```
SELECT
    customer_id,
    COUNT(*) AS total_payments
FROM payment
GROUP BY customer_id;
```

Result:

customer_id	total_payments
1	32
2	27
3	26
...	...

15. AVG with GROUP BY

Find the average payment made by each customer:

```
SELECT
    customer_id,
    ROUND(AVG(amount), 2) AS average_payment
FROM payment
GROUP BY customer_id;
```

For more details about the PostgreSQL built-in `ROUND()` function, see the [ROUND\(\) Function](#) section.

16. MIN and MAX with GROUP BY

Find the minimum and maximum payment for each customer:

```
SELECT
    customer_id,
    MIN(amount) AS minimum_payment,
    MAX(amount) AS maximum_payment
FROM payment
GROUP BY customer_id;
```

17. GROUP BY with Multiple Aggregate Functions

We can combine them:

```
SELECT
    customer_id,
    COUNT(*) AS total_payments,
    SUM(amount) AS total_amount,
    ROUND(AVG(amount), 2) AS average_amount,
    MIN(amount) AS minimum_amount,
    MAX(amount) AS maximum_amount
FROM payment
GROUP BY customer_id;
```

This gives a complete payment summary for every customer.

18. HAVING — Filtering Groups

WHERE filters **rows**.

HAVING filters **groups**.

For example:

Show customers whose total payments are greater than 100.

```
SELECT
    customer_id,
    SUM(amount) AS total_payment
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 100;
```

Important difference

```
WHERE → filters individual rows
HAVING → filters grouped results
```

19. WHERE vs HAVING

WHERE

```
SELECT
    customer_id,
    SUM(amount) AS total_payment
FROM payment
```

```
WHERE amount > 5
GROUP BY customer_id;
```

First, payments below or equal to 5 are removed.

Then customers are grouped.

HAVING

```
SELECT
    customer_id,
    SUM(amount) AS total_payment
FROM payment
GROUP BY customer_id
HAVING SUM(amount) > 100;
```

First, customers are grouped.

Then groups with total payment ≤ 100 are removed.

20. Aggregate Functions with JOIN

The real power of aggregate functions appears when we combine them with **JOIN**.

For example, instead of showing only **customer_id**, we can show the customer's name.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(p.amount) AS total_payment
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name;
```

Now we can see:

```
Customer Name → Total Payment
```

21. Example: Top Customers

Suppose we want to find the customers who have paid the most.

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(p.amount) AS total_payment
FROM customer c
JOIN payment p
    ON c.customer_id = p.customer_id
GROUP BY
    c.customer_id,
    c.first_name,
    c.last_name
ORDER BY total_payment DESC;
```

The most valuable customers appear first.

22. Important SQL Execution Idea

For beginner students, remember this simplified order:

```
FROM
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
ORDER BY
```

For example:

```
SELECT
    customer_id,
    SUM(amount) AS total_payment
FROM payment
WHERE amount > 2
GROUP BY customer_id
HAVING SUM(amount) > 100
ORDER BY total_payment DESC;
```

Think of it as:

Get data → **Filter rows** → **Make groups** → **Filter groups** → **Display** → **Sort**

23. Quick Reference

Function	What it does	Example
<code>COUNT()</code>	Counts	<code>COUNT(*)</code>
<code>SUM()</code>	Adds values	<code>SUM(amount)</code>
<code>AVG()</code>	Calculates average	<code>AVG(amount)</code>
<code>MAX()</code>	Finds highest value	<code>MAX(amount)</code>
<code>MIN()</code>	Finds lowest value	<code>MIN(amount)</code>

Remember

`COUNT` → How many?
`SUM` → How much in total?
`AVG` → What is the average?
`MAX` → What is the highest?
`MIN` → What is the lowest?

Practice Exercises — dvdrental

Level 1 — Basic Aggregate Functions

Exercise 1

Find the total number of customers.

Expected concept: `COUNT()`

Exercise 2

Find the total number of films.

Table: `film`

Exercise 3

Find the total number of payments.

Table: payment

Exercise 4

Find the total payment amount received.

Expected concept: SUM()

Exercise 5

Find the average payment amount.

Expected concept: AVG()

Exercise 6

Find the highest payment amount.

Expected concept: MAX()

Exercise 7

Find the lowest payment amount.

Expected concept: MIN()

Level 2 — Aggregate + WHERE

Exercise 8

Count how many payments are greater than 5.

Exercise 9

Calculate the total amount of payments greater than 5.

Exercise 10

Find the average payment amount for payments greater than 5.

Exercise 11

Find the highest payment made by customer ID 10.

Exercise 12

Find the total payment made by customer ID 10.

Level 3 — GROUP BY

Exercise 13

Find the number of payments made by each customer.

Hint:

```
GROUP BY customer_id
```

Exercise 14

Find the total payment made by each customer.

Exercise 15

Find the average payment made by each customer.

Exercise 16

Find the minimum and maximum payment made by each customer.

Exercise 17

Find the total number of rentals for each customer.

Table: rental

Exercise 18

Find the number of films for each rating.

Table: film

Hint:

```
GROUP BY rating
```

Level 4 — GROUP BY + JOIN

Exercise 19

Display each customer's:

- Customer ID
- First name
- Last name
- Total payment

Use **customer** and **payment**.

Exercise 20

Find the top 10 customers based on total payment.

Hint:

```
ORDER BY ... DESC  
LIMIT 10
```

Exercise 21

Display each customer's name and number of payments.

Exercise 22

Display each customer's name and average payment.

Exercise 23

Find the total payment collected for each staff member.

Tables:

```
payment  
staff
```

Level 5 — HAVING

Exercise 24

Find customers whose total payment is greater than **100**.

Exercise 25

Find customers who have made more than 30 payments.

Exercise 26

Find customers whose average payment is greater than 4.

Exercise 27

Find film ratings having more than 200 films.

Challenge Exercises

Exercise 28

Find the top 5 customers based on total payment.

Display:

```
customer_id  
first_name  
last_name  
total_payment
```

Exercise 29

Find the total payment collected by each staff member.

Display:

```
staff_id  
first_name  
last_name  
total_payment
```

Exercise 30

Find the average payment for each customer and display only customers whose average payment is greater than 4.

Exercise 31

Find the number of rentals for each film.

Use:

```
film
inventory
rental
```

Think about how the tables are related before writing the query.

Exercise 32 — Final Challenge

Create a **Customer Payment Report** showing:

```
Customer ID
First Name
Last Name
Number of Payments
Total Payment
Average Payment
Minimum Payment
Maximum Payment
```

Sort the report by **Total Payment from highest to lowest**.

This exercise combines:

```
COUNT()
SUM()
AVG()
MIN()
MAX()
JOIN
GROUP BY
ORDER BY
```

Suggested Learning Sequence

1. Why aggregate functions?
↓
2. COUNT()
↓
3. SUM()
↓
4. AVG()
↓
5. MAX()
↓

```
6. MIN()
  ↓
7. Multiple aggregate functions
  ↓
8. Aggregate + WHERE
  ↓
9. GROUP BY
  ↓
10. GROUP BY + multiple aggregates
  ↓
11. HAVING
  ↓
12. JOIN + aggregate functions
  ↓
13. Real-world reports
  ↓
14. Practice exercises
```

Tips to Remember

Aggregate functions turn many rows into useful summary information.

And the most important five are:

COUNT = How many? SUM = How much? AVG = Average? MAX = Highest? MIN = Lowest?